

Summer School on AI for Economics and Finance

Deep surrogates: solve the model once, then estimate, quantify, and design

Simon Scheidegger

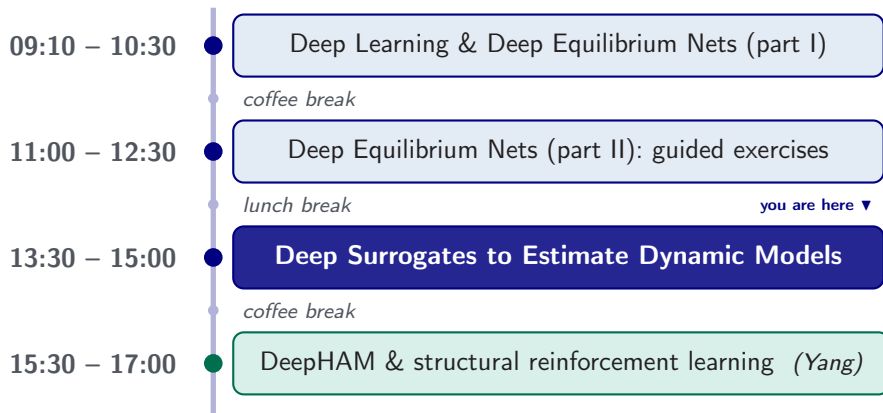
HEC, University of Lausanne

Based on Chen, Didisheim & Scheidegger (2026), *Journal of Financial Economics* 177, 104222

University of Torino

August 24, 2026

Day 1 at a Glance



Materials: github.com/sischei/summer-school-AI-...-2026 · Cloud: nuvolos.cloud

Roadmap of This Lecture (90 min)

I. What Is a Surrogate? (12 min)

- Look-up tables, pseudo-states, and what the speed-up buys

II. Estimate: structural estimation (20 min)

- SMM on a surrogate; Brock–Mirman; the identification ridge

III. Gaussian Processes (24 min)

- When data is expensive, a deep net is the wrong tool
- Kernels, credible bands, and active learning

IV. Quantify: uncertainty (10 min)

- Univariate effects, Sobol' indices, Shapley values for the SCC

V. Design: the optimal carbon tax (16 min)

- Policy coefficients as pseudo-states; 500 solves, not a million

VI. Synthesis and Hands-On (8 min)

- When to use what; the notebooks

Learning Objectives

By the end of this lecture you should be able to:

- ▶ Explain a surrogate as a **differentiable look-up table** and say precisely which classical bottleneck it removes
- ▶ Promote structural or policy parameters to **pseudo-states**, so that one solve answers a whole family of questions
- ▶ Estimate a structural parameter by simulated method of moments on a trained surrogate, and read an identification ridge off the criterion surface
- ▶ Turn the same surrogate into a global sensitivity analysis: univariate effects, Sobol' indices, Shapley values
- ▶ Fit a Gaussian process, read its credible band, and **decide between a GP and a deep network** from the cost of one model evaluation
- ▶ Place the next expensive evaluation where it buys the most, using Bayesian active learning

Part I

What Is a Surrogate?

The Bottleneck Is Always the Same

Expensive to evaluate

One evaluation of a modern structural model means solving a Bellman equation, a PDE, or a Monte Carlo simulation: minutes to hours.

And we almost never want just one evaluation:

- ▶ **Estimation** sweeps thousands of candidate parameter vectors.
- ▶ **Uncertainty quantification** propagates a distribution over parameters.
- ▶ **Policy design** optimizes over a space of policy rules.

The one advantage we have

We own the model. Unlike an econometrician with a fixed sample, we can generate as much training data as we are willing to pay for, and the labels are exact.

Look-up Tables $\rightarrow$ Neural Networks

The old idea: pre-compute once, then interpolate.

x	$\Phi(x)$
-2.0	0.0228
-1.0	0.1587
0.0	0.5000
1.0	0.8413
2.0	0.9772

The normal CDF table in the back of every old statistics textbook. Nobody re-derives Φ ; they look it up.

A 21st-century look-up table:

a network that has memorized $\mathbf{x} \mapsto y$,

$$\phi(\mathbf{x} \mid \mathbf{w}) \approx y = f(\mathbf{x}).$$



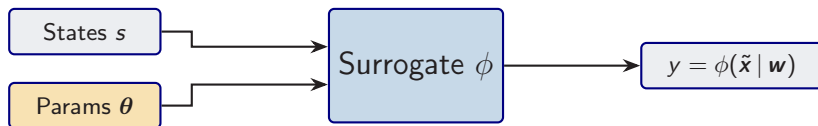
A table has one entry per grid point and dies of the curse of dimensionality. A network is **continuous**, **differentiable**, and its cost grows with the complexity of f , not with n_g^d .

Pseudo-States: The Whole Trick

Chen, Didisheim & Scheidegger (2026)

Do not build one surrogate per parameter vector. Treat the parameters as **extra state variables**:

$$\tilde{\mathbf{x}} = \underbrace{(s_1, \dots, s_n)}_{\text{states}}, \underbrace{(\theta_1, \dots, \theta_p)}_{\text{parameters}} \in \mathbb{R}^d, \quad d = n + p.$$



Solve the model **once** over the augmented space $\Rightarrow$ an instant answer for *any* parameter configuration in the box. Everything in the rest of this lecture is a different question asked of that one object.

Building One, in Four Steps

1. **Design.** Draw $\tilde{\mathbf{x}}_i$, $i = 1, \dots, m$, over the box $\prod_j [\underline{x}_j, \bar{x}_j]$: uniform, Latin hypercube, Sobol' sequence, or (Part III) actively.
2. **Label.** Evaluate the expensive model: $y_i = f(\tilde{\mathbf{x}}_i)$. **This is the step that costs money.**
3. **Train.** Minimize the empirical risk

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \frac{1}{m} \sum_{i=1}^m |\phi(\tilde{\mathbf{x}}_i | \mathbf{w}) - y_i|^2.$$

4. **Validate.** Held-out max and mean absolute error, and for parametric surrogates, error as *a function of θ* , not just on average.

Where the error actually comes from

Because data is unlimited, scarcity-driven overfitting is not the binding constraint. What binds is **approximation error** (network capacity against the true regularity of f) and **label noise** when f is itself an approximate inner solver.

Not Ordinary Supervised Learning

Standard supervised learning

- ▶ Data is scarce and noisy
- ▶ Overfitting is the main enemy
- ▶ Regularization, early stopping, cross-validation
- ▶ Careful tuning, small learning rates

Surrogate modeling

- ▶ Data is **unlimited**: we own the model
- ▶ Labels are **exact** up to solver tolerance
- ▶ Large budgets, aggressive learning rates
- ▶ Focus on *approximation*, not generalization

What a trained surrogate gives you

- ▶ **Accurate enough for the downstream task**: in the option-pricing benchmark, 95th-percentile implied-vol errors below 0.13% of the mean, rising to 0.43% under a week to maturity. **Accuracy is not uniform over the box**
- ▶ **Orders of magnitude faster**, and **differentiable**: gradients in θ free, by autodiff

What the Speed-Up Buys

Table 2 of Chen, Didisheim & Scheidegger (2026), *Journal of Financial Economics*, for the Bates model:

All SPX options on an average day	FFT	Surrogate	+ GPU
Pricing, 1 day ($\approx 4,000$ options)	10 s	0.6 s	0.06 s
Gradient, 1 day	180 s	3.2 s	0.3 s
Estimation, 1 year (252 days $\times$ 10 iterations)	125 h	2.2 h	0.2 h

The point is not the speed but the new question

At 125 hours per pass, re-estimating the model every day for a year was not a slow research project; it was **not a research project at all**. At 0.2 hours it becomes an option-implied tail-risk measure with a daily time series.

Notation from Here On

This is an *estimation and policy* lecture, so one symbol changes meaning relative to this morning.

Symbol	Meaning here	This morning (DEQN deck)
θ	structural parameters (e.g. ϱ, β)	not used
ϑ	policy parameters (tax and transfer rules)	not used
$\mathbf{w}$	network weights	written θ
$\phi(\cdot \mathbf{w})$	the surrogate	$\mathcal{N}_\theta$

Why the swap

Every paper in this literature writes θ for the object being estimated; keeping θ for network weights would make the SMM criterion in Part II unreadable. **The weights are never the object of interest here**; they are an implementation detail of the look-up table.

Three Questions, One Surrogate

The rest of the lecture is the same object, post-processed three different ways.

Operator	Question	Post-processing	Where
Estimate	Which parameters fit the data?	$\arg \min$ of an SMM criterion over θ	Part II
Quantify	Which parameter uncertainties matter?	propagate $p(\theta)$ through a quantity of interest: univariate effects, Sobol', Shapley	Part IV
Design	Which policy is best?	constrained $\arg \max$ of a welfare objective over ϑ	Part V

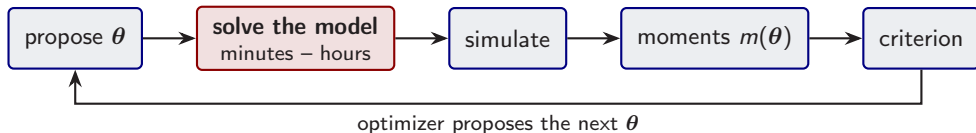
The same skeleton throughout: surrogate 1 solves the equilibrium over states *and* pseudo-states; when the quantity of interest is *itself* expensive, a second surrogate maps parameters to it. **Part II needs only surrogate 1:** its moments are cheap. Parts IV and V need both, and Part III is about what surrogate 2 should be made of.

Part II

Estimate: Structural Estimation

The Estimation Loop, and Why It Is Expensive

The classical nested fixed point. For each candidate θ :



The surrogate move. Put θ *inside* the network at training time. The red box then runs **once**, and the loop becomes a forward pass.

Two consequences, not one

- ▶ The criterion becomes cheap enough to evaluate on a fine grid, so you can *look at* it rather than trust a local optimizer's report.
- ▶ The criterion becomes **differentiable in θ** , because the surrogate is.

SMM: The Moment Condition

Let the data give T observations $\{y_t\}$. Choose moment functions $g(y) \in \mathbb{R}^q$ such that, at the true parameter $\theta_0 \in \mathbb{R}^p$ with $q \geq p$,

$$\mathbb{E}[g(y_t) - m(\theta_0)] = 0, \quad m(\theta) = \mathbb{E}_\theta[g(y_t)].$$

The empirical and simulated counterparts are

$$\hat{g}_T = \frac{1}{T} \sum_{t=1}^T g(y_t), \quad \hat{m}_S(\theta) = \frac{1}{S} \sum_{s=1}^S g(y_s^{\text{sim}}(\theta)).$$

Identification. θ_0 is the *unique* zero of $\mathbb{E}[g(y_t)] - m(\theta)$ on the parameter space.

Just-identified: $q = p$. Over-identified: $q > p$, which buys a testable restriction. It does *not* automatically sharpen the criterion: a moment with a near-zero column in M adds noise and finite-sample bias, as the joint exercise will show.

In this lecture. y_t is log output, consumption, and capital; g collects three volatility and autocorrelation moments; $\theta = \varrho$ first, then $\theta = (\beta, \varrho)$.

SMM: Estimator and Weighting

$$\hat{\theta}_{\text{SMM}} = \arg \min_{\theta} [\hat{g}_T - \hat{m}_S(\theta)]^\top W [\hat{g}_T - \hat{m}_S(\theta)], \quad W \succ 0.$$

Common random numbers: the detail that decides success

Hold the shock draws $\{\varepsilon_s\}$ **fixed across θ** . Then $\hat{m}_S(\theta)$ is a smooth function of θ , with a clean minimum. Re-draw them and you are optimizing a noisy surface: the optimizer chases simulation noise and reports wherever it happened to land.

Weighting matrix.

- ▶ **Identity** $W = I$: the honest pedagogical default; consistent, inefficient.
- ▶ **Diagonal** $W = \text{diag}(1/\hat{\sigma}_g^2)$: rescales each moment by its sampling variance.
- ▶ **Optimal** $W^* = \Omega^{-1}$, the inverse asymptotic covariance of the moment gap, proportional to $\hat{\Sigma}_g^{-1}$ up to the simulation factor $1 + 1/S$. Minimum asymptotic variance. ($S = 10$ costs 5% in standard errors; $S = 1$ costs 41%.)

SMM: What the Asymptotics Tell You to Look At

Write $M = \partial m(\theta_0)/\partial \theta^\top$ for the moment Jacobian, and Ω for the asymptotic covariance of the **moment gap** $\sqrt{T}[\hat{g}_T - \hat{m}_S(\theta_0)]$. If the simulated shocks are drawn independently of the data, $\Omega = (1 + 1/S)\hat{\Sigma}_g$ with S panels of the data's length; as $S \rightarrow \infty$, $\Omega \rightarrow \hat{\Sigma}_g$. **The inflation is the price of simulating rather than solving.**

Under the usual regularity conditions, as $T \rightarrow \infty$,

$$\sqrt{T}(\hat{\theta}_{\text{SMM}} - \theta_0) \xrightarrow{d} \mathcal{N}(0, V(W)), \quad V(W) = (M^\top W M)^{-1} M^\top W \Omega W M (M^\top W M)^{-1},$$
which collapses to $V(W^*) = (M^\top \Omega^{-1} M)^{-1}$ at the efficient weighting.

Identification, read geometrically

V exists only if M has **full column rank**. A near-rank-deficient M is a **flat direction** of the criterion: an *identification ridge*. Standard errors explode, the point estimate is arbitrary along the ridge, and no amount of data at those moments fixes it.

Because the surrogate makes the criterion cheap, you can **plot the ridge** instead of inferring it from an ill-conditioned Hessian. We do exactly that at the end of this part.

The Surrogate Is Not Free

Everything on the previous slide assumed the moments came from *the model*. They came from a network. Write $\hat{m} = m + e$, where e is the surrogate's approximation error.

e does not shrink with the sample

More data makes $\hat{g}_T$ precise; it does nothing to e . This is **non-local misspecification**: the estimator converges not to θ_0 but to a **pseudo-true** θ^* minimizing the population criterion under $\hat{m}$, with Hall–Inoue asymptotics rather than Hansen's.

$$\sqrt{T}(\hat{\theta} - \theta^*) \xrightarrow{d} \mathcal{N}(0, \Sigma(\mu^*)), \quad \theta^* \rightarrow \theta_0 \text{ only as } \sup_{\theta} \|e(\cdot, \theta)\| \rightarrow 0.$$

The practical consequence, and the whole reason Part I ended with a validation step: report the surrogate's **maximum** error over the parameter box, not its mean. The maximum bounds the bias, and a surrogate with excellent *average* accuracy can still be badly wrong in the one direction you are estimating. That happens at the end of this part.

Chen, Didisheim & Scheidegger (2026), fn. 15, make the point about their own surrogate; the asymptotics are Hall & Inoue (2003).

The Model: Brock–Mirman with Stochastic TFP

The same model class as this morning, with two deliberate changes: δ drops from 1 to 0.10, which destroys the closed form, and ϱ is promoted from a fixed number to an input.

$$\max_{\{C_t\}} \sum_{t=0}^{\infty} \beta^t \mathbb{E}[\ln C_t], \quad K_{t+1} + C_t = Y_t + (1 - \delta)K_t,$$

$$Y_t = z_t K_t^\alpha, \quad \log z_{t+1} = \varrho \log z_t + \sigma \varepsilon_{t+1}, \quad \varepsilon_{t+1} \sim \mathcal{N}(0, 1).$$

Calibration

α	δ	σ	β (scalar exercise)
0.36	0.10	0.04	0.96

Note the δ . The textbook closed form $s^* = \alpha\beta$ needs $\delta = 1$. Here $\delta = 0.10$, so there is *no* analytic policy and the Euler equation must be solved numerically, which is precisely the expensive inner solve the surrogate replaces. Choosing $\delta = 1$ would have made the exercise vacuous.

Why ϱ , and Not β ?

- ▶ β is famously **poorly identified from business-cycle moments**: volatilities and autocorrelations barely move with it. It does shift the *level* of the capital stock, which is the channel we exploit at the end of this part.
- ▶ In practice β is *calibrated* to a target steady-state interest rate. Under log utility with general δ , the steady-state Euler relation is $1/\beta = 1 + \alpha(k^*)^{\alpha-1} - \delta$.
- ▶ ϱ has a **clean, direct mapping** into autocorrelation moments: $\text{Corr}(\log Y_t, \log Y_{t-1})$ and $\text{Corr}(\Delta \log C_t, \Delta \log C_{t-1})$ both move monotonically with persistence.
- ▶ So ϱ is the textbook **sharply identified** scalar, what you want when learning the workflow. Three moments for one parameter is over-identified, which buys a testable restriction; it does *not* automatically sharpen anything.

Notebook 03_03 does the scalar case. Notebook 03_04 then frees β as well, and the pathology shows up immediately.

Step 1: Train the Surrogate with ϱ as a Pseudo-State

Goal

Learn $\phi : (z_t, K_t, \varrho) \mapsto s_t \in (0, 1)$, with $K_{t+1} = (1 - \delta)K_t + s_t Y_t$.

1. A three-input MLP with a *sigmoid* output head, so $s_t \in (0, 1)$ holds by construction: the hard-constraint trick from this morning.
2. Train on the Euler residual with Gauss–Hermite quadrature. ϱ enters both the policy and the transition. Under log utility the notebooks minimize the scale-free *relative* residual, which shares its zero set with the raw one but conditions far better in FP32:

$$\ell = \frac{1}{N} \sum_i \left(\left[\beta C_t^{(i)} \sum_j w_j \frac{1 - \delta + \alpha z_{t+1,j}^{(i)} K_{t+1}^{(i)\alpha-1}}{C_{t+1,j}^{(i)}} \right]^{-1} - 1 \right)^2$$

3. Two phases: uniform sampling over the (z, K, ϱ) box, then simulation-based refinement.
4. Validate by plotting $s(z=1, K, \varrho)$ across ϱ , and ask first how much the policy *should* move with ϱ . Here the honest answer is “not much”, which is worth knowing *before* you diagnose lazy learning.

Step 2: Simulate, and Choose Moments That Identify

Goal

Generate synthetic data at $\varrho_{\text{true}} = 0.90$; three identifying moments, plus a counterexample.

Moment	What it captures
$m_1 = \text{Std}(\Delta \log C_t)$	growth-rate volatility
$m_2 = \text{Corr}(\Delta \log C_t, \Delta \log C_{t-1})$	persistence in consumption growth
$m_3 = \text{Corr}(\log Y_t, \log Y_{t-1})$	persistence in output, the most direct
$m_0 = \text{mean}(s_t)$ (<i>diagnostic</i>)	nearly flat in ϱ , what not to use here

A trap: $\text{Var}(\log z) = \sigma^2/(1 - \varrho^2)$ increases in ϱ , but $\text{Var}(\Delta \log z) = 2\sigma^2/(1 + \varrho)$ decreases.

And be clear what the “data” are: one simulation of the surrogate itself at $\varrho = 0.90$, under the *same* shock path as every candidate. Deliberate: it isolates identification from sampling noise, but the criterion then hits exactly zero. **This validates the machinery, not the estimator’s finite-sample behavior.**

Step 3: Minimize the Criterion

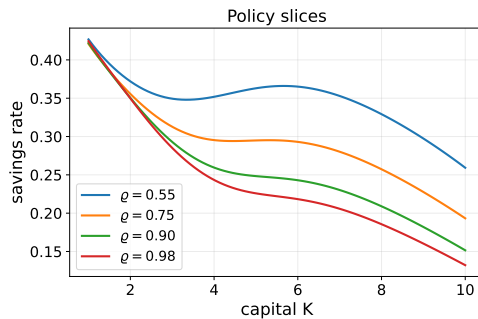
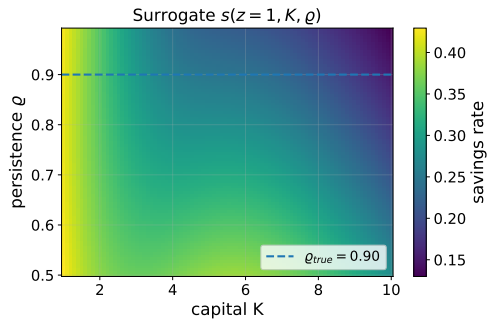
The scalar case, $\theta = \varrho$

$$\hat{\varrho} = \arg \min_{\varrho} [m(\varrho) - \hat{m}^{\text{data}}]^{\top} W [m(\varrho) - \hat{m}^{\text{data}}], \quad W = I.$$

1. One routine $\varrho \mapsto (C_t, I_t, Y_t) \mapsto m(\varrho)$, reusing the trained surrogate throughout.
2. Evaluate on a fine grid over $[0.50, 0.99]$, step 0.005, 99 points. **At surrogate speed this is cheaper than one call to the original model.**
3. Take the grid arg min. No separate optimizer is needed, and you see the whole criterion rather than one path through it.
4. **Check the solution is interior.** A corner solution invalidates the asymptotics we just set up, and usually signals a mis-specified moment.

$W = I$ is inefficient but consistent; the efficient $W^* = \hat{\Sigma}_g^{-1}$ changes the standard errors, not the argument.

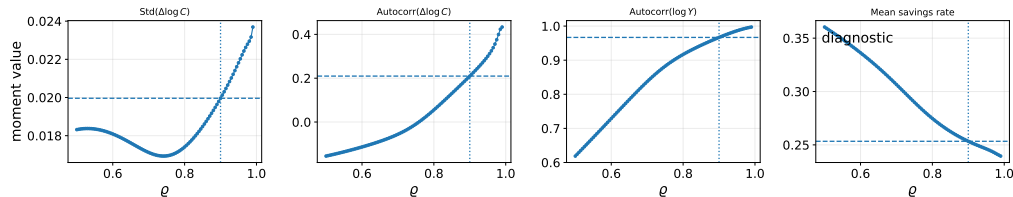
Result: The Surrogate Slice $s(K, \varrho \mid z = 1)$



One trained object standing in for 99 separate model solutions. Note what it does *not* show: the policy depends strongly on K and only **weakly** on ϱ : the four slices differ by only a few per cent at low K , and $\varrho = 0.90$ and 0.98 nearly coincide. That is not a defect. Most of the identifying information sits in the *transition*, not the policy: ϱ sets the persistence of z , and the moments are autocorrelations of simulated paths.

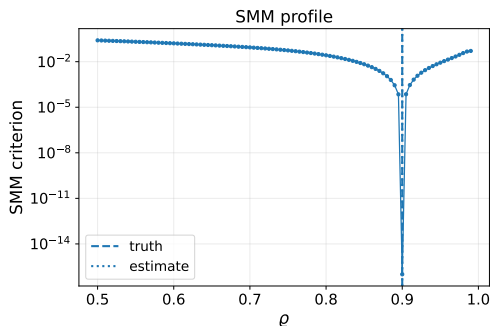
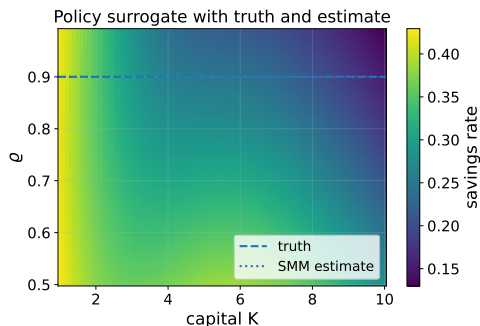
Result: Which Moments Actually Identify

Identifying variation under common random numbers



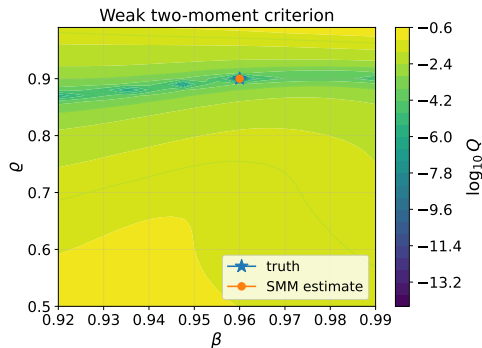
m_2 and m_3 rise monotonically in ρ : either alone would pin the parameter down. m_1 is **U-shaped**: nearly flat below $\rho \approx 0.75$, steep above it, so it identifies well near the truth and barely at all in the lower half of the range. The diagnostic m_0 is **essentially flat**: its measured slope at ρ_{true} is -0.07 , against $+1.02$ for m_2 . A moment with no slope contributes a near-zero column to M , and therefore almost nothing to identification, however well it matches in the data.

Result: The Criterion and the Estimate



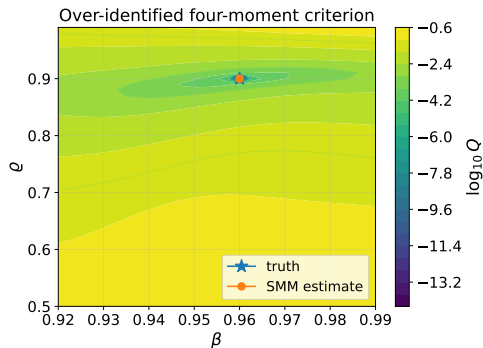
A single sharp interior minimum at $\hat{\rho} = 0.900$, exact because of the design on the previous slide, not because of luck. Note the *log* vertical axis: the criterion falls by three orders of magnitude between $\rho = 0.8$ and the minimum. **Curvature this steep is what good identification looks like**, but with a zero-noise design we cannot turn it into a standard error. Contrast the next slide.

Joint Estimation of (β, ϱ) : A Ridge, and How It Closes



Two dynamic moments. A long valley along β : many (β, ϱ) pairs fit about equally well. **Condition number 96**, smallest singular value 0.021.

The classical result: β moves the capital stock's *level*, so a level moment identifies it and business-cycle moments do not. **Identification is fixed by information, not by a better optimizer.**



Add the level moments. The valley closes into a basin. **Condition number 2.5**, smallest singular value 0.91.

The Ridge That Wasn't

An earlier run of this same notebook produced a ridge that **stayed open** when the level moments were added: condition number 305, the weak direction lying almost exactly along the β axis. It looked like a clean econometric finding. It was not.

The closed form gives $ds^*/d\beta = 1.92$. The under-trained surrogate reported -0.0016 : **near zero and the wrong sign**. Its Euler loss looked fine.

	200 steps	8,000 steps
s at the anchor	0.336	0.254
error vs. $s^* = 0.254$	32%	0.0%
$\partial m_0 / \partial \beta$	-0.0016	+1.27
condition number	305	2.5

The check that catches it

A flat direction is either the moments or **the surrogate**, and the criterion cannot tell you which. Evaluate the surrogate against something known in closed form first: here $s^*(\beta)$ at the steady state. This is *lazy learning* from this morning's Part VI, caught in the act.

From Classroom to Production

Brock–Mirman is the mechanism in miniature. Chen, Didisheim & Scheidegger (2026) run it on a high-dimensional workhorse option-pricing model.

What the surrogate enables

- ▶ Re-estimate **every day** instead of once per paper
- ▶ Systematic out-of-sample evaluation against reduced-form benchmarks
- ▶ Conditional distributions of option returns

What that produced

- ▶ An option-implied **tail-risk index**, predictive of market crashes
- ▶ A **parameter-instability** measure that lines up with option-market illiquidity
- ▶ A read on the structural-versus-reduced-form trade-off

The pattern worth remembering

None of these are faster versions of old results. They are questions whose *answers require a time series of estimates*, and that is exactly what a parametric surrogate manufactures.

Part III

Gaussian Processes:
When Data Is Expensive

When One Label Costs Four Hours

Everything so far assumed labels are cheap. For option pricing they are: 10^6 evaluations is an afternoon. **That fails the moment the model is a global stochastic equilibrium.**

What a deep network needs

- ▶ 10^4 to 10^6 labeled points
- ▶ An architecture and hyperparameters to search for
- ▶ Many epochs of SGD, and no uncertainty unless you train an ensemble

What a Gaussian process needs

- ▶ Tens to a few thousand points
- ▶ A kernel: an explicit smoothness assumption
- ▶ A closed-form posterior, no SGD, and **posterior variance everywhere, for free**

Why the variance is the real prize

It is not decoration: it tells you **where the surrogate does not know the answer**. That is what lets you spend the next expensive evaluation well, and what says whether a welfare surface is trustworthy before you optimize it.

Recall: Multivariate Gaussians

A random vector $\mathbf{x} \in \mathbb{R}^d$ is multivariate Gaussian if

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma),$$

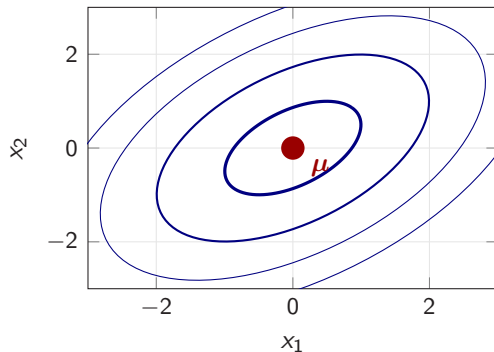
with density

$$p(\mathbf{x}) \propto \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right).$$

Key fact

It is **fully characterized** by $\boldsymbol{\mu}$ and Σ . Nothing else about the distribution is free.

2D Gaussian contours ($\rho = 0.5$)



Conditioning Is Regression

Joint:

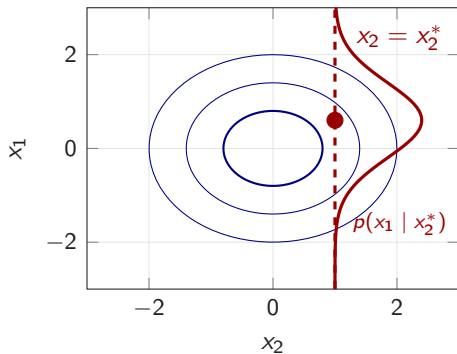
$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}\right)$$

Conditional:

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$

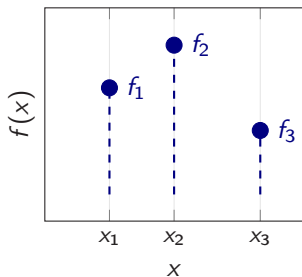
$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

These two lines are the entire engine of GP regression. Everything that follows is bookkeeping about how to build Σ .



Slicing the joint at $x_2 = x_2^*$ leaves a 1D Gaussian: a prediction *and* an error bar.

From Observations to Interpolation



Assume the function values are jointly Gaussian:

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix}\right)$$

Nearby points should be more correlated than distant ones. With a squared-exponential kernel and $\ell = 2$:

$$K = \begin{pmatrix} 1.00 & 0.76 & 0.22 \\ 0.76 & 1.00 & 0.61 \\ 0.22 & 0.61 & 1.00 \end{pmatrix}$$

Build the covariance from a **kernel**, $K_{ij} = k(x_i, x_j)$. Conditioning on the observed f_i then predicts everywhere else.

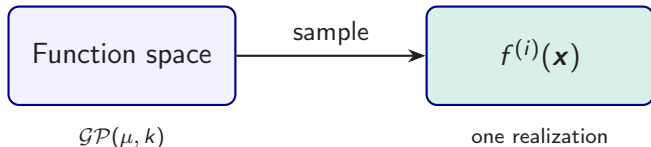
A Gaussian Process: A Distribution over Functions

Definition

A **Gaussian process** is a collection of random variables, any finite subset of which is jointly Gaussian:

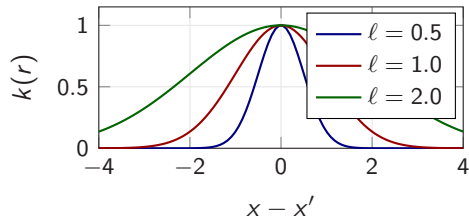
$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')).$$

- ▶ **Mean function** $\mu(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$, almost always 0 after centering the data.
- ▶ **Covariance function** $k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - \mu(\mathbf{x}))(f(\mathbf{x}') - \mu(\mathbf{x}'))]$, where all the modeling lives.



Kernels: Where the Prior Lives

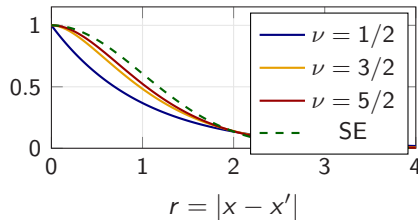
Squared exponential



$k_{\text{SE}}(r) = \sigma_f^2 \exp(-r^2/2\ell^2)$. Larger $\ell \Rightarrow$ smoother; samples are **infinitely differentiable**, often too optimistic.

Choosing the kernel is not a technicality: it *is* the model. Assume more smoothness than the truth has, and the GP interpolates confidently straight through a kink.

Matérn family, $\ell = 1$



ν sets differentiability: samples are $\lceil \nu \rceil - 1$ times differentiable. $\nu = 5/2$ is the **sane default** for value and policy functions.

GP Regression: The Posterior in Two Lines

Joint distribution of training values $\mathbf{f}$ and test values $\mathbf{f}_*$:

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K & K_* \\ K_*^\top & K_{**} \end{pmatrix}\right), \quad K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j).$$

Condition on what you observed, the two lines from three slides ago:

$$\mu_{*|\mathcal{D}} = K_*^\top K^{-1} \mathbf{f}$$

$$\Sigma_{*|\mathcal{D}} = K_{**} - K_*^\top K^{-1} K_*$$

Read the variance line carefully

- ▶ Near the data: $K_*^\top K^{-1} K_* \approx K_{**}$, so the variance **collapses**.
- ▶ Far from the data: $K_*^\top K^{-1} K_* \approx 0$, so the variance **reverts to the prior**.

Note what this means: the GP *knows it is extrapolating*, and says so, without ever seeing the true function out there. No trained network does this by default.

Noise, the Nugget, and Numerical Stability

Observation model. If labels carry noise, $y = f(\mathbf{x}) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma_y^2)$, then $\text{cov}[y_p, y_q] = k(\mathbf{x}_p, \mathbf{x}_q) + \sigma_y^2 \delta_{pq}$, so everywhere replace K by $K_y = K + \sigma_y^2 I$:

$$\bar{f}_* = \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{y}, \quad \sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{k}_*.$$

Noiseless ($\sigma_y^2 = 0$)

- ▶ Interpolates the training points exactly
- ▶ Variance is zero at training inputs
- ▶ Needs K strictly positive definite

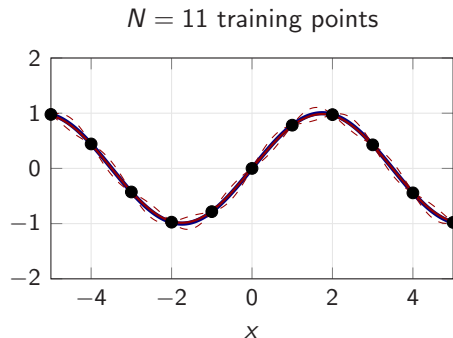
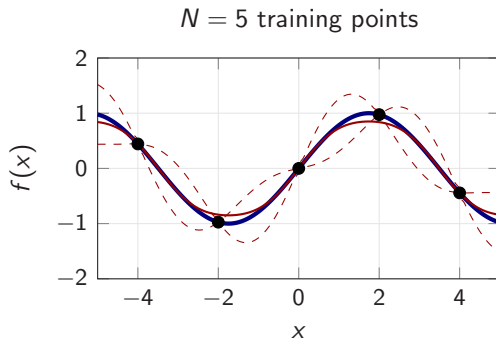
Noisy ($\sigma_y^2 > 0$)

- ▶ Smooths through the training points
- ▶ Variance positive everywhere
- ▶ Much better conditioned

In practice

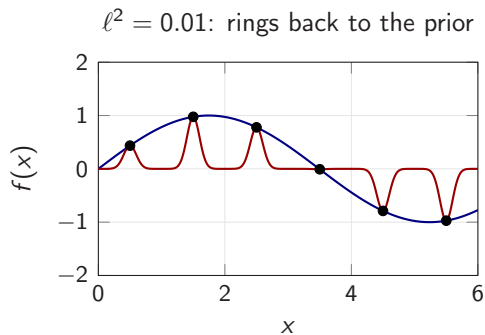
Even for a “noiseless” surrogate, add a jitter of $\sigma_y^2 \sim 10^{-6}$. **And be honest about whether it really is noiseless:** if the labels came from a Monte Carlo simulation or an iterative solver stopped at a tolerance, they carry noise, and pretending otherwise makes the Cholesky factorization fail before it makes the statistics wrong.

Prediction and Credible Bands

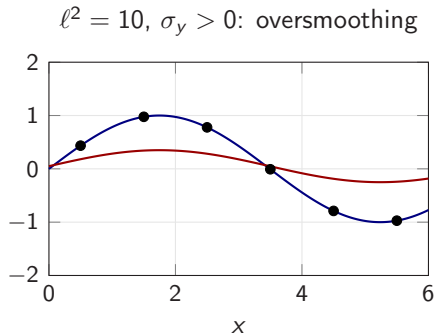


Blue: true function. **Red:** posterior mean. **Dashed:** approximate 95% credible band. The band pinches shut at each training input and balloons between them, and **that shape is the map of where to sample next**, and we use it at the end of this part.

Getting the Length Scale Wrong



Too short. The GP interpolates every point exactly and then falls back to the prior mean in between. Perfect training fit, useless predictions, and the credible band (previous slide) balloons in exactly those gaps.



Too long. The posterior mean flattens and no longer passes through the data. The band is narrow and confidently wrong, which is the more dangerous failure of the two.

Learning the Hyperparameters, and a Free Diagnostic

Marginal likelihood: how ℓ , σ_f , σ_y get chosen

$$\log p(\mathbf{y} \mid X) = \underbrace{-\frac{1}{2} \mathbf{y}^\top K_y^{-1} \mathbf{y}}_{\text{data fit}} \underbrace{-\frac{1}{2} \log |K_y|}_{\text{complexity penalty}} \underbrace{-\frac{N}{2} \log(2\pi)}_{\text{constant}}$$

The two terms pull against each other: **an automatic Occam's razor**, with closed-form gradients and no validation split spent. Optimize with restarts; it is multimodal.

Leave-one-out, for free

$$\mu_{-i}(\mathbf{x}_i) - y_i = -\frac{[K_y^{-1} \mathbf{y}]_i}{[K_y^{-1}]_{ii}}, \quad \sigma_{-i}^2(\mathbf{x}_i) = \frac{1}{[K_y^{-1}]_{ii}}.$$

These need $\text{diag}(K_y^{-1})$, so *all* N leave-one-out residuals cost one further $\mathcal{O}(N^3)$ solve, the same order you already paid, and **no refitting and no held-out data**. Doing N honest refits would be $\mathcal{O}(N^4)$. **This is the number to report for a surrogate**; Part V quotes it.

Closed form: Rasmussen & Williams (2006), §5.4.2, in ../readings.

GP Regression in Fifteen Lines

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

def kernel(X1, X2, l=1.0, sf=1.0):
    sq = np.sum(X1**2, 1).reshape(-1, 1) \
        + np.sum(X2**2, 1) - 2 * X1 @ X2.T
    return sf**2 * np.exp(-0.5 * sq / l**2)

K = kernel(X_tr, X_tr) + 1e-8 * np.eye(N)      # jitter
L, low = cho_factor(K, lower=True)             #  $O(N^3)$ , once
alpha = cho_solve((L, low), y_tr)              # the weights

Ks = kernel(X_tr, X_te)
mu = Ks.T @ alpha                             # posterior mean
v = cho_solve((L, low), Ks)
var = kernel(X_te, X_te) - Ks.T @ v           # posterior variance
```

No training loop, no learning rate, no architecture: the whole model is the kernel and one Cholesky factorization. Compare Algorithm 2.1 in Rasmussen & Williams (2006); notebook 03_02 builds this and checks it against sklearn.

What It Costs: $\mathcal{O}(N^3)$

Cost of exact GP inference

- ▶ **Fitting:** Cholesky of $K_y \in \mathbb{R}^{N \times N}$: $\mathcal{O}(N^3)$ time, $\mathcal{O}(N^2)$ memory.
- ▶ **Prediction:** $\mathcal{O}(N)$ per test point (mean), $\mathcal{O}(N^2)$ (variance).
- ▶ **Hyperparameters:** $\mathcal{O}(N^3)$ per step.

If you really need large N

Sparse GPs (inducing points, rank $M \ll N$, cost $\mathcal{O}(NM^2)$), random Fourier features, GPyTorch.

But notice where this bites

The cubic cost sounds fatal until you ask *why* N is small. If a label costs a four-hour solve, you were never reaching $N = 10^4$ anyway. The regime where GPs are cheap to fit is the regime where data is expensive to get.

Gaussian Processes versus Deep Networks

Criterion	Gaussian process	Deep neural network
Uncertainty	✓built in (posterior variance)	✗needs ensembles or MC dropout
Hyperparameters	✓marginal likelihood	●search: random, Hyperband
Scaling in N	✗ $\mathcal{O}(N^3)$, practical limit $\sim 10^4$	✓ $\mathcal{O}(N)$ per epoch
Scaling in d	✗degrades past $d \approx 10$	✓handles $d \gg 100$
Small data ($N < 100$)	✓excellent	✗underfits, or memorizes
Fitting	✓closed form	●SGD, many epochs, seed-dependent
Interpretability	✓the kernel <i>is</i> the assumption	✗black box

The decision rule is the *cost of a label*, not taste

- ▶ **Labels cheap, d large:** **deep network**, as in Part II's option-pricing surrogate.
- ▶ **Labels expensive, $d \lesssim 10$:** **GP plus active learning**, as in Parts IV and V. (The table's "degrades past $d \approx 10$ " is the same boundary.)
- ▶ **Both, layered:** a DEQN solves the equilibrium, a GP maps its output to the QoI.

Finger Exercise: How Many Points?

PAUSE: 5 min

The setting

You must map a welfare objective over a **6-dimensional** policy box. One evaluation means simulating a stochastic OLG model, ≈ 4 hours of wall-clock time. You have a 64-core machine and **two weeks**.

1. Roughly how many evaluations can you afford?
2. How many points per dimension would a tensor grid give you at that budget?
3. Which approximator does that budget select, and why?

Solution: How Many Points?

1. The budget

14 days $\times$ 24 h = 336 h of wall-clock time; at 64 cores that is $\sim 21,500$ core-hours, so about **5,000 evaluations** at 4 h each. Call it a few thousand, allowing for failures and re-runs.

2. What a grid does with them

$5,376^{1/6} \approx 4.2$ points per dimension. Four per axis will not resolve a curved welfare surface, and refining to a still-inadequate six costs $6^6 = 46,656$ evaluations = 187,000 core-hours, or **four months** on the same 64 cores (21 years on one).

3. Which approximator

A few thousand points is far too few to train a deep network, and comfortably inside the $\mathcal{O}(N^3)$ range of an exact GP. **Choose the GP**, and since every point is precious, do not place them on a grid at all. Let the posterior variance choose them.

The real study in Part V spent **500**, not 5,000. The next three slides are how.

Where to Sample Next

Fixed design

- ▶ Spread evenly, fixed before you know anything
- ▶ Wasted where the function is flat

Active design

- ▶ Goes where the surrogate is **uncertain**
- ▶ Reuses the posterior variance you already have

The score function

$$U(\mathbf{x}) = w_{\text{obj}} \mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$$

- ▶ $\sigma^2(\mathbf{x})$, **exploration**: sample where the variance is large.
- ▶ $\mu(\mathbf{x})$, **objective-following**: for Bayesian *optimization*.

In practice Part V uses the upper-confidence-bound form $\alpha = \lambda\mu + \kappa\sigma$, $\lambda = 1$, $\kappa = 100$: exploration-weighted, but not pure, because there the optimum *is* the target. **And note what σ^2 does not depend on**: with hyperparameters fixed it is a function of the *design* alone, for any kernel. Refit ℓ each step and the data creep back in; two slides on, that matters.

The BAL Algorithm

Bayesian active learning for surrogates

Require: initial design $\mathcal{D}_0 = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n_0}$, budget N , candidate set $\mathcal{C}$

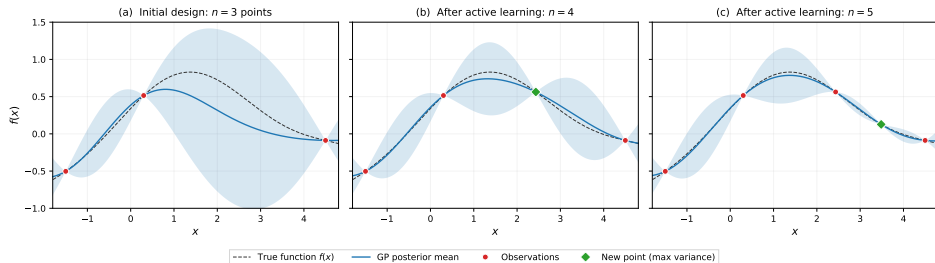
- 1: fit a GP on $\mathcal{D}_0$, giving $\mu(\cdot)$ and $\sigma^2(\cdot)$
- 2: **for** $t = 1, \dots, N - n_0$ **do**
- 3: evaluate $U(\mathbf{x}) = w_{\text{obj}} \mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{C}$
- 4: select $\mathbf{x}_{\text{new}} = \arg \max_{\mathbf{x} \in \mathcal{C}} U(\mathbf{x})$
- 5: **query the expensive model:** $y_{\text{new}} = f(\mathbf{x}_{\text{new}})$
- 6: update $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(\mathbf{x}_{\text{new}}, y_{\text{new}})\}$ and refit the GP
- 7: **end for**
- 8: **return** the trained GP surrogate

Scheidegger & Bilonis (2019), in ../readings, puts this loop to work on economic models.

Every step except line 5 is cheap; the loop exists to make line 5 happen as rarely as possible, and in a real study to tolerate its failure.

BAL in Action

Each new point goes where the posterior variance is largest (the marked point).



(a) three observations, the posterior drifting where there is no data. (b) the acquisition takes the max-variance point. (c) one more fills the gap: **five points suffice**.

But do not oversell it. Notebook 03_02 runs this against a uniform design and they finish *level* (MAE 0.0234 vs 0.0237 at $N = 25$). For a *stationary* kernel the posterior variance depends on the design alone, so on an interval uncertainty sampling essentially *is* a space-filling design. It earns its keep where no such design exists: **irregular domains, ARD kernels, and dimensions where a grid is impossible**.

Part IV

Quantify: Uncertainty

How Uncertain Is the Social Cost of Carbon?

- ▶ Carbon taxation is the consensus policy response, and the level of the tax is anchored on the **social cost of carbon** (SCC): the marginal welfare loss from one extra tonne of CO₂.
- ▶ The SCC is computed inside **integrated assessment models** (IAMs), coupling an economy to a climate system.
- ▶ Those models carry serious **parametric** uncertainty: the equilibrium climate sensitivity, the damage function, risk aversion, the discount rate, on top of stochastic tipping risk.

The question, and the wall

Not “what is the SCC?” but “**which uncertainties actually drive it?**”, because that tells a modeler where to spend the next year of work and a policymaker which number to scrutinize. Answering it globally means propagating a distribution over parameters through the model. One-at-a-time perturbations are *local* and depend entirely on where you happened to start.

Friedl, Kübler, Scheidegger & Usui (2023), *Deep Uncertainty Quantification*, in ../readings.

Uncertain Parameters as Pseudo-States

Step 1: DEQN

$$P(x_{\text{state},t}, \vartheta)$$

Step 2: GP(θ)

$$\widehat{\text{QoI}}(\vartheta)$$

Step 3: Optimizer

$$\arg \max \widehat{\text{QoI}}(\vartheta)$$

1. **Surrogate 1.** A DEQN solves the IAM globally, with the uncertain θ carried as pseudo-states, the move Part II made with ϱ .
One model solve, not one per draw.
2. **Simulate.** Draw N parameter vectors, simulate under the learned policies, record the quantity of interest, $\text{SCC}(\theta)$.
3. **Surrogate 2.** Fit a cheap approximation to $\theta \mapsto \text{SCC}(\theta)$ from those N points.
4. **Post-process.** Univariate effects, Sobol' indices, Shapley values, millions of queries of surrogate 2, at zero cost.

Step 3 is where Part III pays off. N is a few hundred: each point is a full model simulation. Far too few for a deep network, comfortably inside the range of an exact GP. **Surrogate 2 here is a Gaussian process.**

Global Sensitivity Analysis: Three Quantities

Let the quantity of interest be $y = Q(\theta_1, \dots, \theta_p)$, with θ distributed over its uncertainty range.

Univariate effect: *direction*

$$\mathcal{M}_i^1(t) = \mathbb{E}_{\theta_{-i}}[y \mid \theta_i = t]$$

How y moves as θ_i sweeps its range, averaging over the rest. Answers “*which way?*”

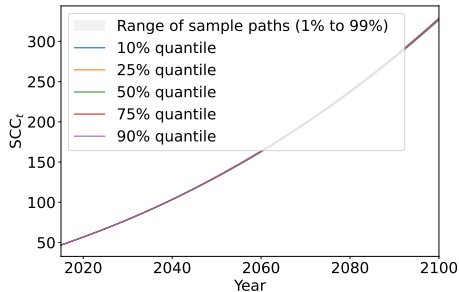
Sobol' index: *prioritization*

$$S_i = \frac{\text{Var}_{\theta_i}[\mathbb{E}_{\theta \setminus \theta_i}[y \mid \theta_i]]}{\text{Var}_{\theta}[y]} \in [0, 1]$$

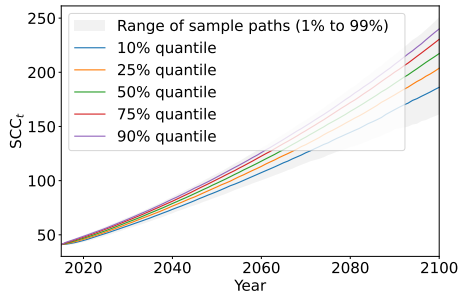
The variance share removed if θ_i were known. The *total* index S_i^T adds every interaction it takes part in. **Screening rule: fix θ_i only if $S_i^T < 1\%$:** a small first-order S_i is not enough, since a parameter can matter entirely through interactions.

Shapley value (*attribution*): splits the variance fairly under interactions, so shares sum to one; first-order Sobol' need not. **The results two slides on are Shapley values.**

The Quantity of Interest: SCC in $\$/\text{tonC}$



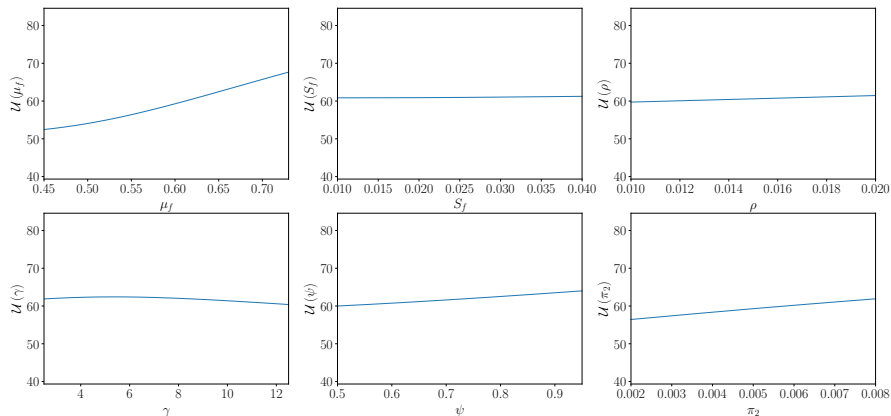
without Bayesian learning



with Bayesian learning

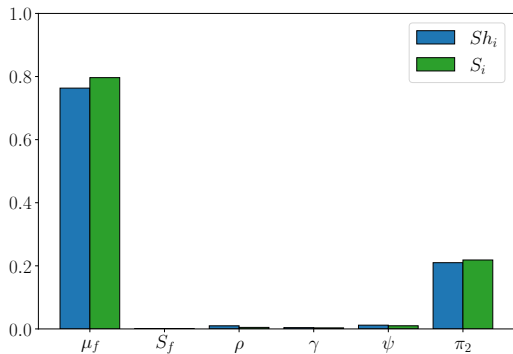
The SCC is not a number, it is a **random variable**. Without learning the quantiles nearly coincide: beliefs are fixed, so the path is near-deterministic. With Bayesian updating they **fan out**: beliefs become a stochastic state and the SCC inherits it. **Learning adds dispersion rather than removing it.**

Univariate Effects on the SCC in 2020

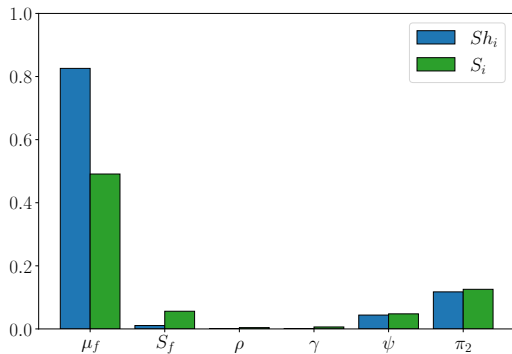


Each panel sweeps one uncertain parameter across its range, integrating out the rest. **Flat panels are parameters you may as well calibrate**; steep and *curved* panels are where the answer depends on a value nobody has pinned down, and where the next modeling effort should go.

What Drives the Variance: Shapley and Sobol'



no learning



with learning

Shapley values and first-order Sobol' indices for the SCC in 2020. The ranking is the deliverable: it says which of the deeply contested parameters the answer is actually sensitive to, and which of the arguments in the literature are, for this purpose, beside the point.

UQ Takeaways, and the Turn to Part V

- ▶ SCC uncertainty is driven almost entirely by the prior belief about the **climate feedback factor** $\mu_{f,0}$ (an ECS of $\approx 3.4^\circ\text{C}$ at the benchmark, and $\approx 80\%$ of the Shapley mass read off the chart) and by the **damage curvature** π_2 . Over the ranges considered, the discount rate, risk aversion and the EIS contribute **almost nothing**.
- ▶ That uncertainty **resolves in roughly a decade**, which itself changes optimal policy today.
- ▶ The whole analysis was post-processing: **the equilibrium was solved once**.

From describing to deciding

Everything so far has been *descriptive*: which parameters fit the data, which uncertainties matter. **Part V asks which policy is best**: same pipeline, same GP with active learning, but the operator becomes a **constrained** arg max, with 40 cohort welfares as constraints.

Part V

Design: The Optimal Carbon Tax

The Design Problem

Why this is hard economics

- ▶ Carbon pricing is the consensus instrument, but with **overlapping generations** a simple tax creates winners and losers.
- ▶ The losers are the *currently alive*, and they vote. Optimal and politically dead is not a policy.
- ▶ So the object is a **constrained** optimum: maximize welfare *subject to no cohort being made worse off*.

This is the wall. Parts I to IV were built to get through it: pseudo-states so the equilibrium is solved once, a GP so the welfare surface is mapped from a few hundred points, and active learning so those points are well spent.

Kübler, Scheidegger & Surbek (2026), in ../readings.

Why this is hard computation

- ▶ A 12-period stochastic OLG IAM with stochastic carbon intensity and a stochastic tipping point.
- ▶ Expected welfare at *one* policy is 10,000 forward simulations, about *two seconds*. Cheap, until you notice the constrained search needs **millions** of queries, of the objective *and* of 40 cohort-welfare surfaces.
- ▶ The Pareto constraint is on **40 cohort welfares**, not one aggregate.

Policy Coefficients as Pseudo-States

The move, one more time

The tax is a *rule*, linear in cumulative emissions,

$$\tau_t(E_t) = \vartheta_0 + \vartheta_E E_t,$$

with revenue recycled through a sharing rule. The coefficients ϑ are **embedded as pseudo-states in the DEQN**, as ϱ was in Part II and θ in Part IV.

Surrogate 1: the equilibrium

- ▶ DEQN over the $d_x = 17$ state *plus* the policy pseudo-states: 2, 14, then **16** across the three scenarios, a **33-dimensional** solve at the top end
- ▶ Several hours from scratch, minutes warm-started

Cross-reference to this morning: the pathology here is *lazy learning*: low residual, nearly flat in ϑ .
Cure: anchors and distillation, Part VI of the DEQN deck.

Surrogate 2: the welfare surface

- ▶ Maps $\vartheta \mapsto \mathbb{E}[\mathcal{W}(\vartheta)]$, plus one GP *per cohort* for the constraints
- ▶ Built from surrogate 1, and it is what we optimize

The Welfare Surrogate: 500 Solves

1. The mapping

$\vartheta \mapsto \mathbb{E}[\mathcal{W}(\vartheta)]$, a Monte Carlo mean of lifetime utilities.

One training point = 10,000 simulated paths.

2. The design

450 by *uniform* sampling over the policy box (the DEQN's own design), then 50 by active learning.

Total 500, rising to 800 for the Pareto scenarios.

3. The model

Squared exponential with automatic relevance determination.

Leave-one-out error 3.2×10^{-5} , against a stopping threshold of 10^{-4} .

Why ARD earns its keep: one length scale per dimension makes the fitted kernel *report* which policy coefficients the welfare surface responds to: a long length scale means that one barely matters. A free sensitivity analysis of the policy space, delivered with the fit.

And note what 500 replaces. Mapping the surface by brute force means a grid: even 4 points per axis is 4^d evaluations, each one 10,000 simulated paths. At $d = 2$ that is affordable; by $d = 16$ it is 4.3×10^9 . The GP reconstructs the surface from a few hundred well-placed points either way.

Three Policy Scenarios

► **Scenario 1: unconstrained welfare maximization.**

Linear tax on cumulative emissions, transfers fixed by an exogenous sharing rule.

$$\tau_t(E_t) = \vartheta_0 + \vartheta_E E_t$$

► **Scenario 2: Pareto-improving policy.**

Same tax rule, but the **transfers are now optimized** subject to no cohort losing.

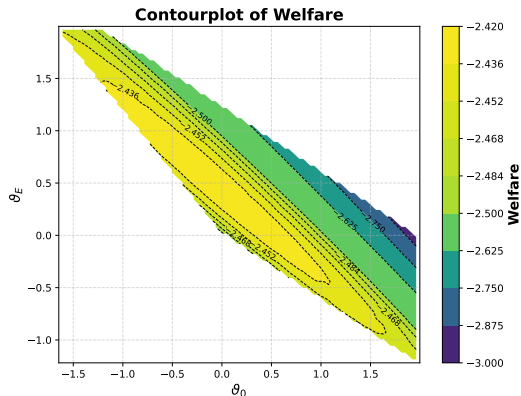
$$\tau_t(E_t) = \vartheta_0 + \vartheta_E E_t, \quad \text{transfer shares} \in \vartheta$$

► **Scenario 3: more instruments.**

Tax also on carbon intensity κ_t and on proximity to the tipping point, with optimal transfers.

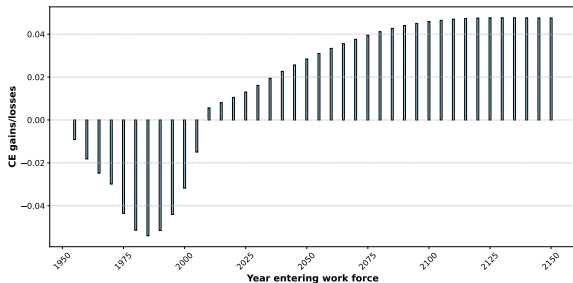
$$\tau_t = \vartheta_0 + \vartheta_E \frac{E_t}{E_0} + \underbrace{\vartheta_\kappa \frac{\kappa_t}{\kappa_0}}_{\text{intensity}} + \underbrace{\vartheta_{TP}(1 - \mathbb{D}_{TP})}_{\text{tipping risk}}$$

Scenario 1: The Welfare Surface



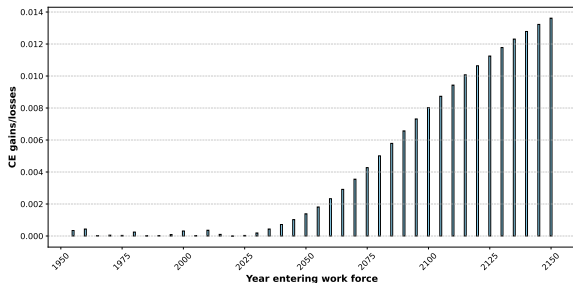
- ▶ The GP posterior mean of social welfare over the tax intercept ϑ_0 and slope ϑ_E .
- ▶ The optimum is the interior peak. Finding it is now a **constrained arg max on a smooth surface**, the same operator as Part II's arg min, opposite sign of intent.
- ▶ Constraints keep the implied tax rate inside $[0, \bar{\tau}]$ at both ends of the horizon.
- ▶ The surface is a picture, not an optimizer's report, exactly the advantage a cheap surrogate buys.

Scenario 1: The Aggregate Gain, and Who Pays for It



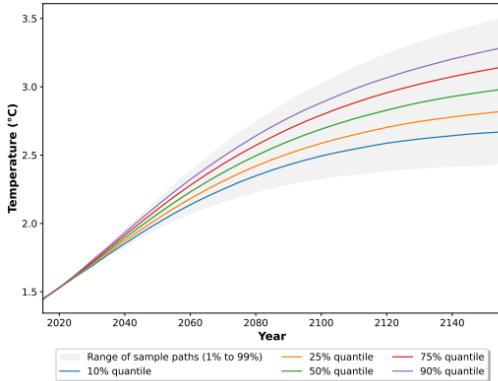
- ▶ **Aggregate welfare** rises by **1.6%** in consumption-equivalent terms.
- ▶ Cohorts entering the workforce from about 2100 on gain up to **4.8%**.
- ▶ **X But:** every cohort that entered before ≈ 2007 (everyone working today) loses, by up to **5%**.
- ▶ **Conclusion:** without transfers, the welfare-maximizing carbon tax is **politically dead on arrival**, and no amount of aggregate-welfare arithmetic changes that.

Scenario 2: Buying Consensus with Transfers



- ▶ **The constraint:** transfers are optimized so that initial generations, and those born over the next decade, stay at **business-as-usual welfare**: zero loss, by construction.
- ▶ **Future gains:** flat at zero for cohorts alive today, then rising steadily to $\approx 1.4\%$ by 2150.
- ▶ **The price of consensus:** smaller than the unconstrained 4.8%, but **no generation loses**.
- ▶ **Aggregate:** **0.42%**, about a quarter of the unconstrained gain.

What the Policy Actually Does



- **Mean warming** settles just below **3 °C**, *higher* than unconstrained.
- **The 99th percentile never exceeds 3.5 °C**, against a much fatter tail under business as usual.
- It buys not a lower average but a **truncated tail**.
- **Reading:** the tax works as **disaster insurance**, not aggressive mitigation.
- **Caveat:** the tipping threshold is confined to [2.5, 3.5] °C by construction, so this says the policy pushes the distribution *off* the upper barrier.

Scenario 3: Does Policy Complexity Pay?

Adding instruments means adding pseudo-states: the policy space grows from $d_{\vartheta} = 14$ to **16** (4 tax parameters plus 12 transfer shares). At the optimum,

$$\{\vartheta_0, \vartheta_E, \vartheta_{\kappa}, \vartheta_{TP}\} = \{-0.237, 0.203, 0.037, 0.012\}.$$

Policy specification	Instruments	Aggregate welfare gain
Linear in cumulative emissions E_t	2	0.42%
Linear in E_t, κ_t, TP_t	4	0.45%

Diminishing returns

A simple, well-designed tax on cumulative emissions captures **93%** of the achievable welfare gain. The two extra instruments (taxing carbon intensity and proximity to the tipping point) buy almost nothing *once intergenerational participation constraints are imposed*.

Note the qualifier. It is the Pareto constraint, not the physics, that flattens the return to complexity.

The Cost of Political Feasibility

1. Welfare maximizing (unconstrained)

- ▶ Aggressive early mitigation
- ▶ Warming stabilizes near 2.7°C
- ▶ $\times$ Current generations lose up to 5%
- ▶ Aggregate gain 1.6%

2. Pareto improving (constrained)

- ▶ Slower initial mitigation path
- ▶ Warming stabilizes near 3.0°C
- ▶ $\checkmark$ No generation loses
- ▶ Aggregate gain 0.42%, tail cut

The trade-off, stated plainly

Securing political support requires delaying some mitigation, costing about $+0.3^{\circ}\text{C}$ in mean warming. In exchange it eliminates the extreme-damage scenarios, insuring future generations against disasters rather than shaving the average.

None of this was visible before the surrogate. The comparison requires optimizing a 16-dimensional constrained problem whose objective needs 10,000 simulations per query, which is to say, it requires everything in Parts I to IV.

Part VI

Synthesis and Hands-On

One Surrogate, Three Questions

	Part II: estimate	Part IV: quantify	Part V: design
Model	Brock–Mirman	stochastic IAM with Bayesian learning about the climate sensitivity	stochastic OLG IAM with tipping risk
Surrogate 1 inputs	(z, K, β, ϱ)	climate-economy states plus uncertain θ	OLG state plus 16 policy pseudo-states
Qol	moments m_1 to m_3	SCC(θ)	40 cohort welfares
Surrogate 2	none needed	GP + active learning	40 GPs + active learning
Operator	arg min under common random numbers	global sensitivity: univariate effects, Sobol', Shapley	constrained arg max
Output Headline	an interior estimate a flat direction is the moments <i>or</i> the surrogate	an importance ranking climate sensitivity and damages drive the SCC	a Pareto-improving tax 0.42% gain, and the tail is cut

The Toolbox: When to Use What

Method	Use when	Watch out for
Polynomials, splines	$d \leq 3$, smooth, cheap labels	no kinks, no irregular domains
Sparse grids	$d \lesssim 20$, local features matter	tensor structure, hypercube domain
Gaussian process	labels expensive, $d \lesssim 10$	$\mathcal{O}(N^3)$; the kernel <i>is</i> the model
Deep network	labels cheap, d large	no uncertainty; architecture search
GP + active learning	every evaluation is precious	sequential, so hard to parallelize

A decision procedure, in order

1. **Price one label.** Seconds, or hours? That number decides more than anything else here.
2. **Count the dimensions** *after* promoting parameters to pseudo-states, not before.
3. **Ask whether you need an error bar.** If a decision rides on the surrogate, you do.
4. **Layer them** when model and quantity of interest have different cost profiles, the common case.

Practical Guidance

- ▶ **Train once, then simulate for free.** Surrogate 1 costs hours from scratch and minutes warm-started. Every downstream question is then a forward pass. Budget accordingly: the expensive part is the design, not the training.
- ▶ **Validate *in the parameter direction*, not on average.** A surrogate with excellent mean error can still be nearly flat in θ : low residual, wrong policy. This is **lazy learning**; the cure (anchors and distillation) is Part VI of this morning's deck. **Do not skip this check**; it is the most common way a surrogate study goes quietly wrong.
- ▶ **Report leave-one-out error, not training error**, and report the **maximum** over the parameter box, not the mean. The maximum is what bounds the bias in θ^* .
- ▶ **Use common random numbers whenever you sweep a parameter.** Without them you are optimizing simulation noise, in estimation *and* in policy design.
- ▶ **When one quantity-of-interest evaluation is expensive, add the second surrogate.** Expected welfare at one policy took 10,000 Monte Carlo paths; the GP layer cut the whole study to 500 simulations at leave-one-out error 3.2×10^{-5} .

Hands-On: What to Run

Notebooks in `../code/03_deep_surrogates`: PyTorch + scikit-learn, CPU-only, self-contained. 03_01 and 03_02 ship on `RUN_MODE = "smoke"`; raise it once a run works.

- ▶ **A deep surrogate, end to end** (≈ 3.5 min). Black–Scholes over (S, K, T, σ, r) , then implied-vol inversion by root-finding *and* by gradient descent. **Read its timing table:** on this toy the surrogate *loses* to numpy: the “price one label first” rule biting. 03_01
- ▶ **GP regression and active learning** (≈ 30 s). A GP from scratch in numpy, checked against sklearn; then the BAL loop and its budget sweep. 03_02
- ▶ **Estimating ϱ by SMM** (≈ 10 s). Train with ϱ as a pseudo-state, simulate under common random numbers, recover the parameter. 03_03
- ▶ **The identification ridge** (≈ 10 s). Free β as well, watch the valley appear, and read the moment Jacobian’s singular values. 03_04

If you only run one: 03_02. It builds a GP from scratch, and its budget sweep is the measurement behind the honest version of the active-learning claim.

Questions?

Simon Scheidegger

HEC, University of Lausanne

simon.scheidegger@unil.ch · sischei.github.io



Next up, 15:30 – 17:00: *Solving heterogeneous agent models with DeepHAM and structural reinforcement learning* (Yang), where the model whose parameters we just estimated becomes one with a full cross-sectional distribution.

Materials: github.com/sischei/summer-school-AI-2026

Cloud: nuvolos.cloud